

.Net Minimal API PhoneBook Backend Knowledge Bytes

<https://gemini.google.com/app/1e18b7c791a526c6>

User prompt: namespace Phonebook.Dtos; /// <summary>A row skipped during import. "Row" is the file row number (header = row 1, first data row = row 2).</summary> public record ContactImportError(int Row, string Reason);

Response: Byte 1: The Import Error Diagnostic Record

Builds on: None — starting point

In plain terms: When a batch import skips invalid rows, the caller needs an exact breakdown of what went wrong. This lightweight data carrier ties the specific row number in the uploaded file to a human-readable explanation of the failure.

The code:

```
namespace Phonebook.Dtos;

/// A row skipped during import. "Row" is the file row number (header = row 1, first data row = row 2).
public record ContactImportError(int Row, string Reason);
```

This positional C# record encapsulates a single failure report within an import response. Using 1-based index numbering where row 1 represents the CSV/spreadsheet header, Row pinpoints the offending line for the user (e.g., row 2 for the first record), while Reason contains the exact validation or conflict message (e.g., "Duplicate phone number or email").

User prompt: namespace Phonebook.Dtos; /// <summary>Body of POST /contacts/import.</summary> public class ContactImportRequest { public List<ContactCreateRequest>? Rows { get; set; } }

Response: Byte 1: The Batch Import Payload Container

Builds on: None — starting point

In plain terms: When uploading a batch of contacts (for instance, parsed from a CSV file), the API needs a root envelope object to hold the collection of incoming rows.

The code:

```
namespace Phonebook.Dtos;

/// Body of POST /contacts/import.
public class ContactImportRequest
{
    public List? Rows { get; set; }
}
```

This class acts as the top-level Data Transfer Object (DTO) bound to the POST /contacts/import request body. It wraps a nullable list of individual ContactCreateRequest records under the property Rows, which matches the JSON payload structure ({ "rows": [...] }) before downstream validation inspects the list's size and contents.

User prompt: namespace Phonebook.Dtos; public record ContactImportResponse(int Imported, int Skipped, List<ContactImportError> Errors);

Response: Byte 1: The Import Result Summary Contract

Builds on: None — starting point

In plain terms: After executing a bulk import, the server sends back a summary report telling the client how many records succeeded, how many were ignored, and the detailed list of errors for the rows that failed.

The code:

```
namespace Phonebook.Dtos;

public record ContactImportResponse(
    int Imported,
    int Skipped,
    List Errors);
```

This positional C# record models the JSON body returned by POST /contacts/import. It provides immutable, strongly typed properties: Imported tracks successfully persisted contacts, Skipped counts rejected records, and Errors carries the collection of ContactImportError objects indicating the exact row numbers and failure reasons.

User prompt: using System.ComponentModel.DataAnnotations; using Phonebook.Middleware; using Phonebook.Validation; namespace Phonebook.Dtos; /// <summary> /// Query parameters of GET /contacts/ (filters, sort, pagination). The [Range] /// attributes and their hand-

set messages are the ones the old controller declared; /// Bind() converts the raw query string with the rules the MVC model binder used. ///

```
</summary> public class ContactListQuery { public string? Search { get; set; } public bool? Favorite { get; set; } public string? Tag { get; set; }
public bool Unlabeled { get; set; } public bool Recent { get; set; } public string Sort { get; set; } = "name_asc"; [Range(1, int.MaxValue,
ErrorMessage = "must be greater than or equal to 1")] public int Page { get; set; } = 1; [Range(1, 100, ErrorMessage = "must be between 1
and 100")] public int Limit { get; set; } = 10; /// <summary> /// Converts the raw query string, collecting errors in parameter order: /// absent
values keep their defaults; an empty or whitespace-only value means /// "not set" for strings and nullable booleans but is an error for the
others; /// a value that cannot be converted fails the whole request with /// "Invalid request parameter" — the same precedence ASP.NET
Core MVC's /// FormatException rule had, regardless of where the bad parameter sits; /// remaining errors (empty values, [Range]) are
joined into one detail string. /// </summary> public static ContactListQuery Bind(IQueryCollection query) { var errors = new
List<RequestError>(); var result = new ContactListQuery(); result.Search = NullIfWhitespace(Raw(query, "search")); var favorite =
Raw(query, "favorite"); if (!string.IsNullOrEmpty(favorite)) { if (bool.TryParse(favorite, out var value)) { result.Favorite = value; } else {
throw new BadRequestException("Invalid request parameter"); } } result.Tag = NullIfWhitespace(Raw(query, "tag")); var unlabeled =
Raw(query, "unlabeled"); if (unlabeled is not null) { if (unlabeled.Length == 0) { errors.Add(new RequestError(nameof(Unlabeled), "The value
" is invalid.")); } else if (bool.TryParse(unlabeled, out var value)) { result.Unlabeled = value; } else { throw new BadRequestException("Invalid
request parameter"); } } var recent = Raw(query, "recent"); if (recent is not null) { if (recent.Length == 0) { errors.Add(new
RequestError(nameof(Recent), "The value " is invalid.")); } else if (bool.TryParse(recent, out var value)) { result.Recent = value; } else {
throw new BadRequestException("Invalid request parameter"); } } var sort = Raw(query, "sort"); if (!string.IsNullOrEmpty(sort)) {
result.Sort = sort; } var page = Raw(query, "page"); if (page is not null) { if (page.Length == 0) { errors.Add(new RequestError(nameof(Page),
"The value " is invalid.")); } else if (!int.TryParse(page, out var value)) { throw new BadRequestException("Invalid request parameter"); } else
{ result.Page = value; RequestValidation.ValidateProperty(result, nameof(Page), errors); } } var limit = Raw(query, "limit"); if (limit is not null)
{ if (limit.Length == 0) { errors.Add(new RequestError(nameof(Limit), "The value " is invalid.")); } else if (!int.TryParse(limit, out var value)) {
throw new BadRequestException("Invalid request parameter"); } else { result.Limit = value; RequestValidation.ValidateProperty(result,
nameof(Limit), errors); } } if (errors.Count > 0) { throw new BadRequestException(ErrorResponses.Build(errors)); } return result; } ///
<summary>First value of a repeated parameter wins; an absent parameter is null.</summary> private static string? Raw(IQueryCollection
query, string name) => query.TryGetValue(name, out var values) && values.Count > 0 ? values[0] : null; /// <summary> /// MVC converted
whitespace-only query values to null for string targets (real /// values are never trimmed — only all-whitespace ones are dropped). ///
</summary> private static string? NullIfWhitespace(string? value) => string.IsNullOrEmpty(value) ? null : value; }
```

Response: Byte 1: The Query Parameter Model & Default State

Builds on: None — starting point

In plain terms: When clients request a paginated, filtered list of records via GET /contacts, they supply multiple optional query parameters in the URL. A dedicated query class groups these parameters into a single object with safe, sensible default values.

The code:

```
namespace Phonebook.Dtos;

public class ContactListQuery
{
    public string? Search { get; set; }
    public bool? Favorite { get; set; }
    public string? Tag { get; set; }
    public bool Unlabeled { get; set; }
    public bool Recent { get; set; }

    public string Sort { get; set; } = "name_asc";

    [Range(1, int.MaxValue, ErrorMessage = "must be greater than or equal to 1")]
    public int Page { get; set; } = 1;

    [Range(1, 100, ErrorMessage = "must be between 1 and 100")]
    public int Limit { get; set; } = 10;
}
```

This model defines filter flags, search criteria, and pagination boundaries. If the caller does not supply sorting or pagination parameters, the model safely defaults to sorting by name ascending ("name_asc"), starting on Page 1, and limiting results to 10 contacts per page (Limit = 10).

Byte 2: Extraction and Whitespace Sanitization Helpers

Builds on: Byte 1

In plain terms: HTTP query strings can be missing, repeated, or filled with empty whitespace. Small helper methods normalize these edge cases before parsing begins.

The code:

```
/// First value of a repeated parameter wins; an absent parameter is null.
private static string? Raw(IQueryCollection query, string name) =>
    query.TryGetValue(name, out var values) && values.Count > 0 ? values[0] : null;

///
/// MVC converted whitespace-only query values to null for string targets (real
/// values are never trimmed — only all-whitespace ones are dropped).
///
private static string? NullIfWhitespace(string? value) =>
    string.IsNullOrEmpty(value) ? null : value;
```

Raw checks the `IQueryCollection` for a key and returns the first value if repeated (e.g., `?tag=work&tag=home` evaluates to "work"), or `null` if the key was omitted. `NullIfWhitespace` treats empty or blank string inputs as `null`, ensuring pure whitespace queries do not trigger unintended database filters.

Byte 3: Text and Boolean Binding Rules

Builds on: Byte 1, Byte 2

In plain terms: Different query parameter types react differently to empty or invalid values. A missing or blank optional filter simply means "do not filter," but a malformed boolean like `?favorite=maybe` must immediately fail the entire request.

The code:

```
result.Search = NullIfWhitespace(Raw(query, "search"));

var favorite = Raw(query, "favorite");
if (!string.IsNullOrEmpty(favorite))
{
    if (bool.TryParse(favorite, out var value))
    {
        result.Favorite = value;
    }
    else
    {
        throw new BadRequestException("Invalid request parameter");
    }
}

result.Tag = NullIfWhitespace(Raw(query, "tag"));
```

String filters like `search` and `tag` are passed through `NullIfWhitespace`. For nullable booleans like `favorite`, whitespace is ignored, but any non-boolean string causes `bool.TryParse` to return false, immediately triggering a `BadRequestException("Invalid request parameter")`.

Byte 4: Non-Nullable Boolean Flags & Empty Value Errors

Builds on: Byte 1, Byte 2, Byte 3

In plain terms: Unlike nullable booleans, non-nullable query flags (like `?unlabeled` or `?recent`) treat an empty assignment (`?unlabeled=`) as an explicit invalid value error, while still checking that non-empty text parses to true or false.

The code:

```
var unlabeled = Raw(query, "unlabeled");
if (unlabeled is not null)
{
    if (unlabeled.Length == 0)
    {
        errors.Add(new RequestError(nameof(Unlabeled), "The value '' is invalid."));
    }
    else if (bool.TryParse(unlabeled, out var value))
    {
        result.Unlabeled = value;
    }
    else
    {
        throw new BadRequestException("Invalid request parameter");
    }
}
```

If the parameter is present in the query string (`unlabeled is not null`), an empty value (`?unlabeled=`) records a formatted validation failure ("The value '' is invalid.") into the errors list. If non-empty text fails boolean conversion (e.g., `?unlabeled=yes`), it halts the request immediately with "Invalid request parameter".

Byte 5: Numeric Parsing and Range Enforcement

Builds on: Byte 1, Byte 2, Byte 4

In plain terms: Numeric pagination fields (`page` and `limit`) require a two-stage check: first verifying that the text is an integer, and then evaluating whether that number fits within acceptable minimum and maximum bounds.

The code:

```
var page = Raw(query, "page");
if (page is not null)
{
    if (page.Length == 0)
    {
        errors.Add(new RequestError(nameof(Page), "The value '' is invalid."));
    }
    else if (int.TryParse(page, out var value))
    {
        result.Page = value;
    }
    else
    {
        throw new BadRequestException("Invalid request parameter");
    }
}
```

```

    }
    else if (!int.TryParse(page, out var value))
    {
        throw new BadRequestException("Invalid request parameter");
    }
    else
    {
        result.Page = value;
        RequestValidation.ValidateProperty(result, nameof(Page), errors);
    }
}

```

If page is empty (?page=), it records an invalid value error. If it cannot parse into an integer (e.g., ?page=one), it throws an immediate "Invalid request parameter". If it parses into an int, RequestValidation.ValidateProperty runs the [Range(1, int.MaxValue)] annotation, appending range violation errors to errors if the value is 0 or negative.

Byte 6: Aggregate Error Dispatching

Builds on: Byte 1, Byte 4, Byte 5

In plain terms: Once all parameters have been examined, any accumulated validation errors (such as empty inputs or out-of-range numbers) are bundled together into a single composite error response.

The code:

```

// Inside Bind():
if (errors.Count > 0)
{
    throw new BadRequestException(ErrorResponses.Build(errors));
}

return result;

```

Malformed data type failures halt early with an immediate exception, while structural validation checks (like range bounds and empty fields) collect in the errors list in parameter order. If any issues were collected, ErrorResponses.Build formats them into the standard semicolon-delimited error string and throws a BadRequestException. Otherwise, the populated ContactListQuery is returned to the endpoint.

User prompt: namespace Phonebook.Dtos; /// <summary>Body of GET /contacts/ responses.</summary> public record ContactListResponse(List<ContactResponse> Contacts, int Page, int Limit, long Total);

Response: Byte 1: The Paginated Contact List Envelope

Builds on: None — starting point

In plain terms: When returning a collection of results from a search or listing endpoint, you must return more than just the records themselves. Clients also need pagination context—such as current page, page size, and total matching rows—to render page numbers and navigation controls properly.

The code:

```

namespace Phonebook.Dtos;

/// Body of GET /contacts/ responses.
public record ContactListResponse(
    List Contact,
    int Page,
    int Limit,
    long Total);

```

This positional C# record models the root JSON payload returned by GET /contacts. Contacts holds the current slice of mapped contact items, Page and Limit reflect the requested pagination boundaries, and Total provides the overall count of matching contacts across the entire database to calculate total pages on the client.

User prompt: namespace Phonebook.Dtos; /// <summary>Body of GET /contacts/metrics.</summary> public record ContactMetricsResponse(long Total, long Favorites, long RecentlyAdded, long Unlabeled);

Response: Byte 1: The Metrics Summary DTO

Builds on: None — starting point

In plain terms: Dashboards require high-level operational statistics without the overhead of fetching full contact records. This lightweight data carrier bundles aggregated summary numbers into a single structured response.

The code:

```
namespace Phonebook.Dtos;

/// Body of GET /contacts/metrics.
public record ContactMetricsResponse(
    long Total,
    long Favorites,
    long RecentlyAdded,
    long Unlabeled);
```

This positional C# record models the JSON payload returned by GET /contacts/metrics. It exposes four 64-bit integer (Long) counters: Total (all contacts in the database), Favorites (starred contacts), RecentlyAdded (contacts created within the trailing 7 days), and Unlabeled (contacts with zero associated tags).

User prompt: namespace Phonebook.Dtos; /// <summary>JSON shape of a contact as the frontend sees it (snake_case via serializer policy).</summary> public record ContactResponse(long Id, string Name, string PhoneNumber, string? Email, string? Address, bool IsFavorite, DateTime? LastViewedAt, DateTime CreatedAt, DateTime UpdatedAt, List<TagResponse> Tags);

Response: Byte 1: The Public Contact Representation (DTO)

Builds on: None — starting point

In plain terms: When sending contact details back to a client (whether from a single fetch, create, update, or list query), you should expose a dedicated read-only model rather than your internal database entity. This protects internal database fields and gives clients a predictable, stable contract.

The code:

```
namespace Phonebook.Dtos;

/// JSON shape of a contact as the frontend sees it (snake_case via serializer policy).
public record ContactResponse(
    long Id,
    string Name,
    string PhoneNumber,
    string? Email,
    string? Address,
    bool IsFavorite,
    DateTime? LastViewedAt,
    DateTime CreatedAt,
    DateTime UpdatedAt,
    List Tags);
```

This immutable positional C# record models the outgoing JSON response for contact details. It mirrors the exact property names and types the frontend expects, including nullable fields (Email, Address, LastViewedAt), audit timestamps (CreatedAt, UpdatedAt), and the nested collection of associated TagResponse items. The application's global JSON naming policy converts these PascalCase record properties into snake_case over the wire.

User prompt: using System.ComponentModel.DataAnnotations; namespace Phonebook.Dtos; /// <summary> /// Body of PUT /contacts/{id}. All fields are optional: a null field is /// left unchanged on the existing contact (merge semantics, same as Java). /// </summary> public class ContactUpdateRequest { [StringLength(255, ErrorMessage = "size must be between 0 and 255")] public string? Name { get; set; } [StringLength(50, ErrorMessage = "size must be between 0 and 50")] public string? PhoneNumber { get; set; } [RegularExpression(@"^\s\$|^[\^@|s|+@|^\s|+\$", ErrorMessage = "must be a well-formed email address")] [StringLength(255, ErrorMessage = "size must be between 0 and 255")] public string? Email { get; set; } public string? Address { get; set; } /// <summary>null = keep current tags, empty list = remove all tags.</summary> public List<string>? Tags { get; set; } }*

Response: Byte 1: The Partial Update DTO (Merge Semantics)

Builds on: None — starting point

In plain terms: When modifying an existing contact, clients should only need to send the specific fields they want to change. Every property on this update model is optional, meaning any field left null in the request leaves the existing value on the database record untouched.

The code:

```
namespace Phonebook.Dtos;

///
/// Body of PUT /contacts/{id}. All fields are optional: a null field is
/// left unchanged on the existing contact (merge semantics, same as Java).
///
public class ContactUpdateRequest
{
    // Properties and validation rules follow...
}
```

This class acts as the Data Transfer Object (DTO) bound to the PUT `/contacts/{id}` request body. Unlike a strict HTTP PUT (which usually replaces an entire resource), this endpoint supports "merge semantics"—acting like a partial update where omitted or null JSON properties are ignored by the service layer.

Byte 2: Optional Contact Field Validation

Builds on: Byte 1

In plain terms: Even though fields like name, phone, and email are optional when updating, any value that is supplied must still adhere to database size constraints and format rules.

The code:

```
[StringLength(255, ErrorMessage = "size must be between 0 and 255")]
public string? Name { get; set; }

[StringLength(50, ErrorMessage = "size must be between 0 and 50")]
public string? PhoneNumber { get; set; }

[RegularExpression(
    @"^\s*(|^[^@\\s]+@[^@\\s]+\s)",
    ErrorMessage = "must be a well-formed email address")]
[StringLength(255, ErrorMessage = "size must be between 0 and 255")]
public string? Email { get; set; }

public string? Address { get; set; }
```

None of these properties carry `[Required]` attributes, making them nullable (`string?`). However, if a caller passes a new string, `[StringLength]` prevents exceeding database column boundaries (255 for name/email, 50 for phone). The regex on `Email` accepts empty whitespace strings or well-formed email addresses, while `Address` remains unconstrained to match the database text column.

Byte 3: Three-State Tag Updates (null vs. Empty List)

Builds on: Byte 1

In plain terms: Updating associated tags requires distinguishing between two different client intentions: leaving the existing tags alone versus wiping all tags completely.

The code:

```
/// null = keep current tags, empty list = remove all tags.
public List? Tags { get; set; }
```

By leaving `Tags` nullable and uninitialized (unlike the create DTO which defaults to `new()`), the property supports distinct update states:

- Omitting "tags" or passing null signals to `ContactService` that the contact's existing tags should remain unchanged.
- Passing an empty array (`"tags": []`) instructs the service to clear all associated tags from the contact.
- Passing an array with names replaces the current tags with the provided set.

User prompt: using System.ComponentModel.DataAnnotations; namespace Phonebook.Dtos; /// <summary>Body of PATCH /contacts/{id}/favorite.</summary> public class FavoriteUpdate { /// <summary>Nullable so a missing field produces the same 400 as the Java @NotNull.</summary> [Required(ErrorMessage = "must not be null")] public bool? IsFavorite { get; set; } }

Response: Byte 1: The Toggle Request Contract (Nullable Boolean Validation)

Builds on: None — starting point

In plain terms: When updating a single boolean flag via an API, you must distinguish between the client intentionally sending `false` versus completely omitting the field from the request. Making the property nullable lets validation catch missing data instead of quietly defaulting to `false`.

The code:

```
using System.ComponentModel.DataAnnotations;

namespace Phonebook.Dtos;

/// Body of PATCH /contacts/{id}/favorite.
public class FavoriteUpdate
{
    /// Nullable so a missing field produces the same 400 as the Java @NotNull.
    [Required(ErrorMessage = "must not be null")]
    public bool? IsFavorite { get; set; }
}
```

In C#, a primitive bool cannot be null; if a caller omits "isFavorite" from the JSON body, the parser would silently assign it false. By declaring bool? IsFavorite as nullable, an omitted or null JSON value remains null. The [Required] attribute then detects that absence and triggers a 400 Bad Request with "must not be null", ensuring updates only happen when the client explicitly provides true or false.

User prompt: namespace Phonebook.Dtos; public record TagResponse(long Id, string Name);

Response: Byte 1: The Tag Projection DTO

Builds on: None — starting point

In plain terms: When displaying labels or categories in the UI, the client only needs the tag's unique identifier and its text name. This compact record represents that minimal tag information.

The code:

```
namespace Phonebook.Dtos;

public record TagResponse(long Id, string Name);
```

This positional C# record models a single tag. It serves two purposes across the API: as the item shape inside GET /contacts/tags lists, and as the nested element within ContactResponse.Tags. Its immutable properties (Id and Name) map cleanly to JSON (e.g., {"id": 1, "name": "Work"}) without exposing any database entity internals.

User prompt: how is minimal apis different from .net controllers core backend

Response: The core difference between Minimal APIs and ASP.NET Core Controllers comes down to **ceremony, execution pipeline, and design philosophy**.

Controllers follow an object-oriented, convention-heavy MVC architecture introduced over a decade ago. Minimal APIs (introduced in .NET 6) use a lightweight, functional routing approach that eliminates MVC scaffolding to minimize boilerplate and memory overhead.

Key Architectural Differences

Feature	ASP.NET Core Controllers	Minimal APIs
Paradigm	Object-Oriented / Class-based (inherits ControllerBase)	Functional / Delegate-based (lambdas, static methods)
Discovery	Runtime assembly scanning via reflection	Explicit route registration at startup
Pipeline Overhead	Heavier (Action filters, model binders, metadata providers)	Lightweight (Direct mapping from HTTP pipeline to endpoint)
Startup Footprint	Larger memory usage and slower cold-start	Minimal memory usage, faster cold-start
Native AOT	Limited compatibility (heavy reflection dependency)	Full, first-class Native AOT support
Endpoint Interception	Action / Result / Resource Filters	Endpoint Filters (AddEndpointFilter)
Automatic Validation	Built-in MVC model validation via [ApiController]	Manual / Opt-in (e.g., FluentValidation, explicit helpers)

1. Code Ceremony & Structure

The Controller Approach

Requires a separate class, namespace, constructor injection boilerplate, and action attributes.

```
[ApiController]
[Route("api/[controller]")]
public class ContactsController : ControllerBase
{
    private readonly IContactService _service;

    public ContactsController(IContactService service)
    {
        _service = service;
    }

    [HttpGet("{id:long}")]
    public ActionResult Get(long id)
    {
```

```

        var contact = _service.Find(id);
        return contact is not null ? Ok(contact) : NotFound();
    }
}

```

The Minimal API Approach

Attaches endpoints directly to the routing table using fluent builder delegates. Services are injected directly into the handler method parameters.

```

var app = builder.Build();

app.MapGet("/api/contacts/{id:long}", (long id, IContactService service) =>
{
    var contact = service.Find(id);
    return contact is not null ? Results.Ok(contact) : Results.NotFound();
});

```

2. Under the Hood: Routing & Discovery

- **Controllers (Runtime Scanning):** During startup, `AddControllers()` scans referenced assemblies via reflection to discover classes decorated with `[ApiController]` or ending in `Controller`. At request time, requests pass through the MVC Action Invoker, running through an extensive pipeline of filters, model binders, and action selectors.
- **Minimal APIs (Explicit Graphs):** Routes are mapped explicitly at startup into the endpoint routing table (`MapGet`, `MapPost`). The routing engine generates highly optimized dynamic delegates (or compile-time generated delegates via source generators), skipping MVC-specific layers entirely.

3. Model Binding and Validation

- **Controllers:** The `[ApiController]` attribute automatically inspects `ModelState` after binding `DataAnnotations` (`[Required]`, `[StringLength]`). If invalid, it short-circuits execution and produces a standard 400 Bad Request ProblemDetails response before your action code ever runs.
- **Minimal APIs:** Binding is streamlined and faster, but **does not run automatic DataAnnotations validation out of the box**. Validation must either be handled manually (like the custom validation wrappers in the snippets you reviewed earlier), using `Endpoint Filters`, or via third-party libraries like `FluentValidation`.

4. Cross-Cutting Concerns: Filters

- **Controllers:** Use the traditional MVC filter pipeline:
 - Authorization Filters
 - Resource Filters
 - Action Filters (`IActionFilter`, `ActionFilterAttribute`)
 - Exception Filters
 - Result Filters
- **Minimal APIs:** Use **Endpoint Filters** (`IEndpointFilter` / `.AddEndpointFilter(...)`). They wrap the endpoint invocation similarly to middleware, letting you inspect arguments, short-circuit, or transform the return value before and after the handler executes:

```

app.MapPost("/contacts", (ContactCreateRequest req) => Results.Ok())
    .AddEndpointFilter(async (context, next) =>
    {
        // Pre-execution logic (e.g. validation, logging)
        var result = await next(context);
        // Post-execution logic
        return result;
    });

```

5. Native AOT & Cloud Performance

Because Controllers rely heavily on runtime reflection and dynamic code generation for model binding and controller activation, compiling an MVC app with **Native AOT (Ahead-Of-Time compilation)** is problematic.

Minimal APIs leverage **Request Delegate Source Generators (RDG)**. During compilation, the compiler inspects your endpoints and emits C# code that binds parameters, resolves DI services, and writes responses directly. This results in:

- Near-instant startup times (ideal for AWS Lambda, Azure Functions, or container cold starts).
- Drastically reduced memory usage.
- Zero JIT (Just-In-Time) compilation overhead in native deployments.

When to Choose Which?

- **Choose Minimal APIs when:**
 - Building microservices, serverless functions, or high-throughput APIs where latency, throughput, and cold starts matter.
 - Targeting Native AOT compilation.
 - You prefer organizing code by feature/endpoint slice (`MapGroup("/feature")`) rather than spreading logic across controllers, actions, and filter folders.
 - Building single-file prototypes or lightweight service layers.
 - **Choose Controllers when:**
 - Maintaining large legacy codebases built around MVC conventions.
 - Building full-stack web applications that render MVC server-side Views (Razor) alongside API endpoints.
 - Your architecture relies heavily on custom MVC-specific infrastructure (e.g., complex legacy `IActionFilter` or `IModelBinder` implementations that are painful to migrate).
-